Lab 26 — Endpoint Detection with osquery

You will install osquery, ask SQL questions of the live host, and schedule a query that detects a suspicious process.

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): - `osquery`

Step 1 — Install

```
curl -fsSL https://pkg.osquery.io/deb/pubkey.gpg | gpg --dearmor
-o /usr/share/keyrings/osquery.gpg




echo "deb [arch=amd64 signed-by=/usr/share/keyrings/osquery.gpg]
https://pkg.osquery.io/deb deb main" \




  > /etc/apt/sources.list.d/osquery.list




apt update && apt install -y osquery




osqueryi --version
```

Step 2 — Interactive SQL queries

```
osqueryi --json "SELECT name, pid, uid FROM processes WHERE on_disk = 0;"  # in-memory only
```

```
osqueryi --json "SELECT name, version FROM deb_packages WHERE name LIKE 'openssh%';"
```

```
osqueryi --json "SELECT username, type FROM users WHERE shell != '/usr/sbin/nologin';"
```

```
osqueryi --json "SELECT * FROM listening_ports WHERE address = '0.0.0.0';"
```

Every system fact — process, package, port, user — is a SQL table. This is what makes osquery the open-source EDR primitive.

Step 3 — Detect a "fileless" process

Simulate a process whose binary has been deleted:

```
cp /bin/sleep /tmp/.evil; /tmp/.evil 600 &
```

```
rm /tmp/.evil
```

```
osqueryi --json "SELECT name, pid, path FROM processes WHERE on_disk
= 0;"
```

This is a classic indicator of malware that unlinks itself after launch.

Step 4 — Detect new SUID binaries

```
osqueryi --json "SELECT path, mode FROM file WHERE path LIKE
'/usr/bin/%' AND (mode LIKE '%4%') LIMIT 10;"
```

Step 5 — Schedule the queries (osqueryd)

```
cat > /etc/osquery/osquery.conf <<'EOF'
{
  "options": { "logger_path": "/var/log/osquery",
"schedule_splay_percent": 10 },
  "schedule": {
    "fileless_proc": { "query": "SELECT name, pid FROM processes
WHERE on_disk = 0;", "interval": 60 },
```

```
    "new_listeners": { "query": "SELECT pid, port, address FROM
listening_ports;", "interval": 60 },




    "users": { "query": "SELECT username, uid FROM users;",
"interval": 300 }




  }



}



EOF



mkdir -p /var/log/osquery



systemctl enable --now osqueryd



sleep 70



ls /var/log/osquery



tail -3 /var/log/osquery/osqueryd.results.log
```

The results stream as JSON — feed them into Wazuh, ELK, or Splunk for

centralised EDR/XDR.

Step 6 — File integrity (FIM) with osquery

```
cat >> /etc/osquery/osquery.conf <<'EOF'
,
  "file_paths": { "etc": ["/etc/passwd", "/etc/shadow", "/etc/sudoers"] },
  "exclude_paths": { },
  "events": { "disable_subscribers": [] }
EOF
```

(Use `osqueryctl` to validate JSON before restart in production.)

Free online tools

- **osquery docs** — https://osquery.readthedocs.io
- **Fleet (open source EDR console)** — https://fleetdm.com
- **Velociraptor** — https://docs.velociraptor.app
- **Sysmon for Linux** — https://github.com/Sysinternals/SysmonForLinux

- **Atomic Red Team** (test detections) — https://atomicredteam.io

---

What you learned

- That EDR is fundamentally "ask SQL questions of the endpoint".
- The two highest-signal queries (`on_disk=0`, new listening ports).
- How osqueryd's JSON results plug straight into a SIEM.